MEDIPREP-HOSPITAL PROCEDURE PREPARATION AND READINESS SYSTEM

PROJECT REPORT

Submitted in partial fulfillment of the requirements for the award of the degree of Master of Computer Applications of A P J Abdul Kalam Technological University

Submitted by:

RYNU REJI (SJC25MCA-2052)

MASTER OF COMPUTER APPLICATIONS

ST. JOSEPH'S COLLEGE OF ENGINEERING AND TECHNOLOGY, PALAI
(AUTONOMOUS)

CHOONDACHERRY P.O, KOTTAYAM
KERALA

OCTOBER 2026

ST. JOSEPH' S COLLEGE OF ENGINEERING AND TECHNOLOGY, PALAI

(AUTONOMOUS)

(An ISO 9001: 2015 Certified College)

CHOONDACHERRY P.O, KOTTAYAM

KERALA

CERTIFICATE

This is to certify that the project work entitled **"MediPrep – Hospital Procedure Preparation and Readiness System"** submitted by **Rynu Reji** student of Third semester MCA at **ST. JOSEPH'S COLLEGE OF ENGINEERING AND TECHNOLOGY**, **PALAI** in partial fulfillment for the award of Master of Computer Applications is a bonafide record of the project work carried out by her under our guidance and supervision. This report in any form has not been submitted to any other University or Institute for any purpose.

Prof.Rajeesh A.V
Assistant Professor
(Project Guide)

Prof.Alex Jose
Associate Professor
(Project Coordinator)

Dr. Rahul Shajan
Associate Professor
(HoD-Dept. of CA)

Submitted for the Viva-Voce Examination held on _____

Examiner 1:

Examiner 2:

DECLARATION

I **Rynu Reji** , do hereby declare that the project titled **"MediPrep – Hospital Procedure Preparation and Readiness System"** is a record of work carried out under the guidance **Mr.Rajeesh A.V**, Asst. Professor, Department of Computer Applications, SJCET, Palai as per the requirement of the curriculum of Master of Computer Applications Programme of A P J Abdul Kalam Technological University, Thiruvananthapuram. Further, I also declare that this report has not been submitted, full or part thereof, in any University / Institution for the award of any Degree / Diploma.

Place: Choondacherry — Rynu Reji

Date: — (SJC25MCA-2052)

ACKNOWLEDGEMENT

The success of any project depends largely on the encouragement and guidance of many others. I would like to take this opportunity to express my gratitude to everyone who has supported me in the successful completion of this project.

First and foremost, I thank God Almighty for giving me the wisdom, strength, and guidance to complete this project successfully.

I express my sincere thanks to my parents for their encouragement, love, and support throughout my academic journey and during the development of this project.

I extend my gratitude to **Dr. V. P. Devasia, Principal, St. Joseph's College of Engineering and Technology, Palai,** for providing the necessary facilities and support.

I express my sincere thanks to **Dr. Rahul Shajan, Head of the Department of Computer Applications, SJCET, Palai,** for his encouragement and support.

I am deeply grateful to my project guide, **Mr. Rajeesh AV, Asst.Professor, Department of Computer Applications, SJCET, Palai,** for his valuable guidance, suggestions, and support throughout the development of **MediPrep – Hospital Procedure Preparation and Readiness System.**

I also thank all the teachers and non-teaching staff of SJCET, Palai, for their support and for the knowledge they have imparted to me.

Finally, I express my appreciation to my friends for their valuable suggestions, help, and encouragement during the completion of this project.

DEPARTMENT OF COMPUTER APPLICATIONS

VISION

To emerge as a centre of excellence in the field of computer education with distinct identity and quality in all areas of its activities and develop a new generation of computer professionals with proper leadership, commitment, and moral values.

MISSION

- Provide quality education in Computer Applications and bridge the gap between the academia and industry.
- Promoting innovation research and leadership in areas relevant to the socio-economic progress of the country.
- Develop intellectual curiosity and a commitment to lifelong learning in students, with societal and environmental concerns.

PEO'S (Program Educational Objectives)

1. MCA Graduates will be able to progress career productively in software industry, academia, research, entrepreneurship pursuits, government, consulting firms and other IT enabled services.
2. MCA Graduates will be able to achieve peer-recognition as an individual or in a team by adopting ethics and professionalism, and communicate effectively to excel well in crisis and inter-disciplinary teams.
3. MCA Graduates will be able to continue life-long professional development in computing and in management that contributes in self and societal growth.

PROGRAMME OUTCOME

PO1(Foundation Knowledge): Apply knowledge of mathematics, programming logic, and coding fundamentals for solution architecture and problem-solving.

PO2 (Problem Analysis): Identify, review, formulate, and analyze problems primarily focusing on customer requirements using critical thinking frameworks.

PO3 (Development of Solutions): Design, develop, and investigate problems with an innovative approach for solutions incorporating ESG/SDG goals.

PO4 (Modern Tool Usage): Select, adapt, and apply modern computational tools, such as the development of algorithms, with an understanding of the limitations, including human biases.

PO5 (Individual and Teamwork): Function and communicate effectively as an individual or team leader in diverse and multidisciplinary groups, using methodologies such as agile.

PO6 (Project Management and Finance): Apply the principles of project management, including scheduling and work breakdown structure, and be knowledgeable about finance principles for profitable project management.

PO7 (Ethics): Commit to professional ethics in managing software projects, especially in financial aspects. Learn to use new technologies for cybersecurity and insulate customers from malware.

PO8 (Life-long Learning): Continuously enhance management skills and the ability to learn, keeping up with contemporary technologies and ways of working.

24SJMCA245 MINI PROJECT

Co No	COURSE OUTCOMES	K Level
CO1	Identify a real-life project which is useful to society / industry	K2
CO2	Interact with people to identify the project requirements	K2
CO3	Apply suitable development methodology for the development of the product / project	K3
CO4	Analyse and design a software product / project	K3
CO5	Test the modules at various stages of project development	K3
CO6	Build and integrate different software modules	K3
CO7	Document and deploy the product / project	K3

SYNOPSIS

MediPrep – Hospital Procedure Preparation and Readiness System is a Python-based web application developed to simplify patient preparation and readiness monitoring before hospital procedures. Proper preparation is essential before surgeries, diagnostic tests, and other medical procedures. MediPrep provides a centralized platform for procedure scheduling, preparation checklist tracking, and laboratory report management.

The system is designed with four key user roles: Admin, Doctor, Patient, and Hospital Laboratory Technician. The Admin manages user accounts and system access. Doctors schedule procedures for existing patients, review laboratory reports, monitor checklist completion, and verify preparation requirements. Patients can view scheduled procedures, access preparation instructions, and track their checklist status. Hospital Laboratory Technicians can upload official laboratory reports according to their authorization, while reports from external laboratories are handled by doctors through the designated workflow.

MediPrep uses procedure-specific checklists and predefined rule-based alerts to identify incomplete preparation requirements. It helps doctors monitor patient readiness and identify cases that require further attention before scheduled procedures. By maintaining procedure details, preparation status, and laboratory reports in an organized system, MediPrep reduces dependence on manual record-keeping and supports better coordination among hospital personnel.

The application is developed using Python Flask, MySQL, HTML, CSS, and JavaScript. It provides a user-friendly interface and role-based access to support organized and efficient hospital procedure preparation and readiness monitoring.

TABLE OF CONTENTS

1. Introduction 1

1.1 About the project 2

1.2 Objective of the project 2

1.3 Scope of the project 3

2. Initial investigation and Feasibility study 4

2.1 Initial Investigation 5

2.2 Feasibility study 6

2.2.1 Economic Feasibility 7

2.2.2 Technical feasibility 7

2.2.3 Behavioral feasibility 7

2.2.4 Operational feasibility… 8

3. System Analysis 9

3.1 System Analysis 10

3.2 Existing system 10

3.3 Proposed System 11

3.4 Software Requirement Specification 11

4. Modules 16

4.1 Modules 17

4.1.1 User Management Module 17

4.1.2 Procedure Scheduling Module 17

4.1.3 Patient Pre-Checklist Module 18

4.1.4 Lab Report Management Module 18

4.1.5 Readiness Monitoring&Alert Module 18

5. Flow Diagrams 19

5.1 Use Case Diagram 20

5.2 Class Diagram 21

6. System Design 23

6.1 System Design 24

6.2 InputDesign 24

6.3 OutputDesign 25

6.4 TableDesign.....25

7. Tools and Platform_____29

7.1 Front end.....30

7.2 Back end.....30

7.3 Framework.....30

7.4 Platform.....30

7.5 Database.....30

8. Testing_____31

8.1 Unit Testing.....32

8.2 Integration Testing.....32

8.3 System Testing.....32

8.3.1 Acceptance Testing.....33

8.3.2 Validation Testing.....33

8.3.3 Black Box Testing.....33

8.3.4 White Box Testing.....33

8.4 Sample Test Cases.....34

9. System Implementation_____35

9.1 Implementation.....36

9.2 Problem Statement.....36

9.3 Problem Definition.....37

10. Conclusion_____39

11. Appendices_____40

A. Sample Screens.....41

B. Sample Code.....48

C. Hardware and Software Requirements.....56

D. Bibliography.....57

INTRODUCTION

1. INTRODUCTION

1.1 About The Project

"MediPrep" is a modern, Python-based web application developed to streamline patient preparation and readiness monitoring for hospital procedures. It provides a secure and efficient solution for procedure scheduling, preparation tracking, laboratory report management, and readiness monitoring. Designed to improve accuracy and coordination, the platform simplifies interactions between administrators, doctors, patients, and hospital laboratory technicians.

The application features four core user roles: **Admin, Doctor, Patient, and Hospital Laboratory Technician** — each playing an important role in maintaining the system's efficiency. The **Admin** manages user accounts, while **Doctors** schedule procedures for existing patients, review laboratory reports, and monitor preparation status. **Patients** can view scheduled procedures and follow preparation instructions. **Hospital Laboratory Technicians** can upload official laboratory reports, while reports from external laboratories are handled by doctors through the designated workflow.

By integrating preparation checklists, laboratory report handling, and predefined rule-based alerts, MediPrep helps identify incomplete requirements and cases needing further attention. The system supports organized patient information management, improves coordination among hospital personnel, and contributes to efficient hospital procedure preparation through digital technology.

1.1.1 Objective of The Project

The primary objective of this project is to develop a comprehensive, web-based platform that simplifies patient preparation and readiness monitoring for hospital procedures. MediPrep aims to ensure that patients complete the necessary preparation requirements before scheduled surgeries, diagnostic tests, and other medical procedures. The platform improves efficiency by enabling procedure scheduling, preparation checklist tracking, laboratory report management, and predefined rule-based alerts, while also promoting better coordination, organized record-keeping, and reliable patient readiness monitoring.

1.1.2 Scope of The Project

The scope of this project includes the development of a web-based hospital procedure preparation and readiness system that supports four user roles: Admin, Doctor, Patient, and Hospital Laboratory Technician. The platform facilitates procedure scheduling, preparation checklist management, laboratory report handling, and patient readiness monitoring. It enables administrators to manage user accounts, doctors to schedule procedures and review reports, patients to access preparation instructions, and authorized laboratory technicians to upload official reports. By integrating predefined rule-based alerts and readiness tracking, MediPrep helps improve the organization, reliability, and efficiency of hospital procedure preparation.

The system is designed to provide the following benefits:

- User-friendly data entry screens with simple navigation.
- Minimal training required to use the application.
- Reduced dependence on paper-based records.
- Fast access to scheduled procedure information.
- Efficient tracking of patient preparation progress.
- Reduced duplication of patient information.
- Role-based access to authorized system functions.
- Easy access to previous procedure details and records for future reference.

,

INITIAL INVESTIGATION AND FEASIBILITY STUDY

2.1 INITIAL INVESTIGATION

The initial investigation for this project highlights the growing need for a reliable and user-friendly platform to manage patient preparation and readiness before hospital procedures. Proper preparation is essential to ensure that patients meet the necessary requirements before surgeries and diagnostic tests. Traditional methods of maintaining paper records, laboratory reports, and manual checklists can make it difficult to track patient readiness and identify incomplete requirements. MediPrep aims to overcome these challenges by providing a centralized system for procedure scheduling, preparation tracking, and laboratory report management.

A review of the existing process shows that managing procedure details and preparation requirements separately may lead to missing documents, delays, and communication difficulties. MediPrep brings administrators, doctors, patients, and hospital laboratory technicians together on a single platform. Doctors can schedule procedures for existing patients and review laboratory reports, while patients can access their scheduled procedures and preparation instructions. Authorized laboratory technicians can upload official reports, and reports from external laboratories are handled by doctors through the designated workflow.

From a technical perspective, the project uses Python Flask, MySQL, HTML, CSS, and JavaScript to develop a web-based application. The system includes role-based access, procedure-specific checklists, and predefined rule-based alerts to identify incomplete preparation requirements. By combining these features in one platform, MediPrep aims to reduce manual work, improve the organization of patient information, and support efficient coordination among hospital personnel.

2.2 FEASIBILITY STUDY

System feasibility is the evaluation of a proposed system to determine whether it can be developed and implemented successfully. It helps identify the requirements, limitations, and potential benefits of the system. The feasibility study examines the cost of development, availability of technology, user acceptance, and suitability of the proposed system for its intended purpose.

The purpose of a feasibility study is to evaluate the existing process, identify the need for computerization, and determine whether the proposed system can meet user requirements effectively. It also helps assess the efficient use of available resources, the expected benefits, and the practicality of developing the system within the available time and budget. Conducting this study at an early stage helps identify potential risks and determine whether the project can proceed successfully.

For MediPrep – Hospital Procedure Preparation and Readiness System, the feasibility study evaluates the suitability of the proposed web application for managing procedure scheduling, patient preparation checklists, laboratory reports, and readiness monitoring. It considers the availability of suitable hardware and software, development costs, ease of use, and the system's ability to support hospital preparation activities.

The key types of feasibility considered in this project are:

- Economic Feasibility
- Technical Feasibility
- Behavioral Feasibility
- Operational Feasibility

2.2.1 Economic Feasibility

MediPrep is economically feasible because it uses technologies such as Python, Flask, and MySQL, which help minimize software licensing costs. The application can be developed and tested using an existing computer and commonly available development tools. Since the technologies used are widely available, the overall development and maintenance costs can be kept low. By organizing procedure details, preparation checklists, and laboratory reports in a centralized platform, the system can reduce paperwork, manual effort, and the time required to retrieve information. Therefore, MediPrep provides a cost-effective approach to managing hospital procedure preparation.

2.2.2 Technical Feasibility

From a technical perspective, MediPrep is feasible because it uses established technologies, including Python Flask for backend development, MySQL for database management, and HTML, CSS, and JavaScript for the user interface. These technologies support the development of a web application with role-based access and structured data management. The system maintains patient and procedure details in a database, allowing authorized users to access relevant information efficiently. It can be developed, tested, and maintained using standard computer hardware and software. The availability of development tools and technical documentation also makes implementation easier. Its modular structure allows future improvements when required.

2.2.3 Behavioral Feasibility

MediPrep is designed to provide a simple and user-friendly interface for all its users. Administrators can manage user accounts, doctors can schedule procedures and review laboratory reports, patients can view scheduled procedures and preparation instructions, and authorized hospital laboratory technicians can upload official reports. Separate interfaces and straightforward navigation help users access their required functions with minimal training. The preparation checklists make it easier for patients to understand the requirements they need to complete before a procedure. The organized presentation of information also helps hospital personnel monitor preparation progress. Therefore, the system is expected to be convenient and acceptable to its intended users.

2.2.4 Operational Feasibility

The operational feasibility of MediPrep is supported by its ability to organize the activities involved in hospital procedure preparation. The system allows doctors to schedule procedures, monitor patient readiness, review laboratory reports, and identify incomplete checklist requirements. Patients can access their scheduled procedure details and track their preparation progress, while administrators and authorized laboratory technicians perform their assigned tasks. Predefined rule-based alerts help identify requirements that need attention before a scheduled procedure. Centralized record management reduces dependence on paper-based records and makes procedure-related information easier to access. By improving coordination among users and organizing preparation activities in one platform, MediPrep supports more efficient hospital operations.

SYSTEM ANALYSIS

3.1 System Analysis

System analysis is a systematic process of studying an existing system, identifying its problems, and determining the requirements for developing an improved solution. It involves examining the current workflow, understanding user needs, and defining the functional and technical requirements of the proposed system. This phase helps identify the limitations of manual methods and recommends suitable solutions to overcome them.

In MediPrep – Hospital Procedure Preparation and Readiness System, system analysis focuses on improving the management of procedure scheduling, patient preparation checklists, laboratory reports, and readiness monitoring. It identifies the requirements of administrators, doctors, patients, and hospital laboratory technicians. The analysis provides a foundation for system design and development by defining the necessary features, user responsibilities, and data management requirements.

3.2 Existing System

The existing method of managing hospital procedure preparation often depends on paper records, manual checklists, separate laboratory reports, and communication between hospital personnel. Doctors and staff may need to check multiple records to determine whether a patient has completed the necessary preparation requirements. This can make it difficult to identify missing documents, incomplete tests, or instructions that have not been followed.

Manual tracking may also cause delays in updating patient information and communicating preparation status. Laboratory reports obtained from different sources may be maintained separately, making them harder to organize and review. The absence of a centralized system can increase administrative work and make it difficult to monitor patient readiness before scheduled procedures. Therefore, a computerized system is needed to organize procedure details, preparation requirements, and laboratory reports in one place.

3.3 Proposed System

The proposed system, MediPrep, is a web-based application designed to simplify hospital procedure scheduling and patient readiness monitoring. It provides a centralized platform for managing patient information, scheduled procedures, preparation checklists, and laboratory reports. The system supports four user roles: Admin, Doctor, Patient, and Hospital Laboratory Technician.

The Admin manages user accounts, while doctors schedule procedures for existing patients, review laboratory reports, and monitor preparation progress. Patients can view their scheduled procedures and follow the required preparation instructions. Hospital Laboratory Technicians can upload official laboratory reports according to their authorization, while reports from external laboratories are handled by doctors through the designated workflow.

The system uses procedure-specific checklists and predefined rule-based alerts to identify incomplete requirements that need attention. By organizing information in a centralized database, MediPrep helps reduce manual record-keeping, improve coordination among authorized users, and support more efficient hospital procedure preparation.

3.4 Software Requirement Specification(SRS)

The Software Requirements Specification (SRS) describes the functional and non-functional requirements of **MediPrep – Hospital Procedure Preparation and Readiness System**. It defines the system's features, user roles, operating environment, and expected behavior. The document serves as an important reference for the design, development, implementation, and testing of the application. It helps ensure that the system meets the intended objectives and satisfies the requirements of its users.

The SRS specifies how the system supports procedure scheduling, patient preparation tracking, laboratory report management, and readiness monitoring. It describes the responsibilities of administrators, doctors, patients, and hospital laboratory technicians, along with the functions available to each role. It also outlines the requirements for data management, user authentication, access control, and system reliability. By clearly defining these requirements, the SRS provides a structured foundation for developing a reliable, user-friendly, and efficient hospital procedure preparation system.

A. Scope

The scope of MediPrep includes the following features:

- Management of user accounts and role-based access.
- Scheduling procedures for existing patient records.
- Displaying scheduled procedures on the patient dashboard.
- Maintaining procedure-specific preparation checklists.
- Managing official laboratory reports according to user authorization.
- Reviewing and verifying laboratory reports by doctors.
- Monitoring patient preparation and readiness status.
- Identifying incomplete requirements using predefined rule-based alerts.
- Providing access to relevant procedure and preparation information.

B. Overall Description

❖ Product Perspective

MediPrep is a web-based application developed to organize hospital procedure preparation and readiness monitoring. It uses Python Flask for backend processing, MySQL for database management, and HTML, CSS, and JavaScript for the user interface. The system follows a client-server approach in which users access the application through a web browser.

❖ Product Functions

- Admin Functions: Manage user accounts and maintain system access.
- Doctor Functions: Schedule procedures for existing patients, review laboratory reports, monitor checklist completion, and verify preparation requirements.
- Patient Functions: View scheduled procedures, access preparation instructions, and track checklist requirements.
- Hospital Laboratory Technician Functions: Upload official laboratory reports according to the system's authorization rules.

❖ User Characteristics

- Admin: Responsible for managing user accounts and system access.
- Doctor: Uses the system to schedule procedures, review reports, and monitor patient readiness.
- Patient: Uses the application to view procedure details and follow preparation instructions.
- Hospital Laboratory Technician: Uses the system to upload authorized laboratory reports.

❖ **Constraints**

- The application requires compatible computer hardware and software.
- MySQL must be available for database operations.
- Access to system functions must follow the user's assigned role.
- Patient and laboratory information must be handled appropriately.
- Procedure scheduling and checklist tracking depend on accurate information entered into the system.

❖ **Assumptions and Dependencies**

- Users have access to a computer or compatible device and a web browser.
- The database is available when the application is in use.
- Administrators maintain accurate user account information.
- Doctors and authorized laboratory technicians enter and review relevant information correctly.
- Patients follow the preparation instructions provided for their scheduled procedures.

C. Functional Requirements

❖ **User Authentication and Access Control**

- The system should allow authorized users to log in.
- Users should access functions according to their assigned roles.
- The system should restrict unauthorized access to patient and procedure information.

❖ **Patient Management**

- The Admin should be able to create patient accounts.
- Patient information should be stored and retrieved from the database.
- Scheduled procedures should be linked to the correct patient record using the patient's database ID.

❖ **Procedure Scheduling**

- Doctors should be able to select existing patients and schedule procedures.
- The system should store the relevant procedure details.
- Scheduled procedures should be displayed on the corresponding patient's dashboard.

❖ **Preparation Checklist Management**

- The system should provide checklists according to the scheduled procedure.
- Checklist items may include fasting completion, blood test reports, consent forms, ECG completion, and medication instructions.
- The system should display the status of preparation requirements.

❖ **Laboratory Report Management**

- Authorized Hospital Laboratory Technicians should be able to upload official laboratory reports.
- Doctors should be able to review and verify reports.
- Reports obtained from external laboratories should be handled by doctors through the designated workflow.

❖ **Readiness Monitoring and Alerts**

- The system should display patient preparation progress.
- Predefined rules should identify incomplete preparation requirements that need attention.
- Authorized users should be able to review the readiness information and related alerts.

D. Non-Functional Requirements

❖ **Performance**

- The system should respond efficiently to normal user requests.
- Database operations should retrieve and update information accurately.
- The application should support its intended usage without unnecessary delays.

❖ **Usability**

- The interface should be simple and user-friendly.
- Navigation should be clear and consistent across pages.
- Users should be able to access their required functions with minimal training.

❖ **Security**

- Access to system functions should be controlled by user roles.
- Patient information and laboratory reports should be accessible only to authorized users.
- Authentication and data handling should follow appropriate security practices.

❖ **Reliability**

- The system should store procedure and patient information accurately.
- Database relationships should maintain the correct association between patients and scheduled procedures.
- The application should handle incomplete or invalid inputs appropriately.

❖ **Maintainability**

- The application should be organized so that individual components can be updated when necessary.
- The database and application code should support troubleshooting and future maintenance.

E. User Interface Requirements

- The application should provide a clear and consistent interface.
- Each user role should have access to its relevant dashboard and functions.
- Forms should use understandable labels and suitable input fields.
- Procedure details, checklist status, and laboratory report information should be

presented clearly.
- Error messages and status indicators should help users understand the current state of their activities.

F. System Architecture

❖ **System Design**

MediPrep follows a client-server architecture. Users interact with the application through a web browser. The Flask backend processes requests and communicates with the MySQL database to store and retrieve information.

- Frontend: HTML, CSS, JavaScript.
- Backend: Python Flask.
- Database: MySQL.
- Access Interface: Web browser.

❖ **Data Flow**

1. User Interaction: The user logs in and accesses the functions permitted by their role.
2. Request Processing: The Flask application receives the request and performs the required processing.
3. Database Operations: Relevant patient, procedure, checklist, or report information is retrieved or updated in MySQL.
4. Response Generation: The application returns the processed information to the user's interface.
5. Readiness Monitoring: The system displays checklist progress and predefined alerts when requirements need attention.

G. Conclusion

This Software Requirements Specification defines the main requirements of **MediPrep – Hospital Procedure Preparation and Readiness System**. It describes the system's user roles, procedure scheduling, preparation checklist management, laboratory report handling, and readiness monitoring features. The successful implementation of these requirements will provide an organized approach to managing patient preparation information, improve coordination among authorized hospital personnel, and support the monitoring of preparation requirements before scheduled procedures.

MODULES

4.1 MODULES

In a software project, a module is a separate unit of functionality that performs a specific task within the system. Modules help divide the project into manageable parts, making the software easier to develop, maintain, and understand. Each module focuses on a particular function while working together with other modules to achieve the overall objectives of the system.

In MediPrep – Hospital Procedure Preparation and Readiness System, the modules are designed to manage user accounts, schedule hospital procedures, track patient preparation requirements, and handle laboratory reports. The system is divided into the following modules based on its main functionalities.

4.1.1 User Management Module

The User Management Module handles user accounts, login, authentication, and role-based access for Admin, Doctor, Patient, and Hospital Laboratory Technician. It ensures that users can access only the functions permitted for their roles. The Admin creates and manages user accounts, including patient accounts, to maintain organized user records.

a) The system should provide a login facility for authorized users.
b) Each user should be authenticated before accessing the respective dashboard.
c) Role-based access should restrict users to the functions permitted for their roles.
d) The Admin should have the provision to create and manage user accounts.
e) The system should maintain centralized user records for better organization.
f) The system should provide a secure logout option for users.

4.1.2 Procedure Scheduling Module

The Procedure Scheduling Module allows doctors to schedule surgeries, diagnostic tests, and other hospital procedures for existing patients. It maintains procedure details, including the procedure name, scheduled date, time, and related preparation requirements. Scheduled procedures are linked to the corresponding patient record so that the details appear on the correct patient's dashboard.

a) The system should allow doctors to select existing patients and schedule procedures.
b) The system should maintain procedure details, including the procedure type, date, and time.
c) Each scheduled procedure should be linked to the correct patient record using the patient's database ID.
d) The system should display scheduled procedures on the patient's dashboard.
e) The system should maintain procedure records for future reference.
f) The doctor should be able to view scheduled procedures and their preparation status.

4.1.3 Patient Preparation Checklist Module

The Patient Preparation Checklist Module helps patients and doctors monitor the completion of requirements before a scheduled procedure. The checklist contains procedure-specific preparation instructions and requirements. It helps doctors identify incomplete tasks and determine whether further preparation is required.

a) The system should display preparation checklists according to the scheduled procedure.

b) The checklist may include fasting completion, blood test reports, consent forms, ECG completion, and medication instructions, depending on the procedure.

c) Patients should be able to view their preparation requirements and track checklist status.

d) Doctors should be able to monitor checklist completion and verify preparation requirements.

e) The system should display the status of completed and incomplete requirements.

f) The system should generate predefined rule-based alerts when critical preparation requirements remain incomplete.

4.1.4 Laboratory Report Management Module

The Laboratory Report Management Module manages patient laboratory reports. Authorized Hospital Laboratory Technicians can upload official reports, while doctors can review and verify them. Reports from external laboratories are handled by doctors.

a) The system should allow authorized laboratory technicians to upload reports.

b) Reports should be linked to the correct patient.

c) Doctors should be able to review and verify reports.

d) The system should maintain organized report records.

e) Report access should be restricted according to user roles.

f) The system should help identify missing required reports.

4.1.5 Readiness Monitoring and Alert Module

The Readiness Monitoring and Alert Module helps doctors track patient preparation status and identify incomplete requirements before scheduled procedures. It uses predefined rules to display readiness status and alert doctors when critical requirements are incomplete.

a) The system should monitor checklist completion.

b) The system should identify incomplete critical requirements.

c) The system should display patient readiness status.

d) The system should alert doctors about incomplete requirements.

e) Doctors should be able to review alerts and take necessary action.

FLOW DIAGRAMS

5.1 Use Case Diagram

A Use Case Diagram is a UML diagram that shows how users (actors) interact with a system and its functions (use cases). It provides a high-level view of system requirements and scope.

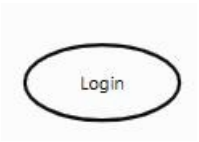
- Represents actors (users or external systems) and the use cases they interact with.
- Shows relationships between users and system functionalities, making requirements easier to understand.

Use Case Diagram Symbols

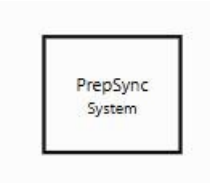
- **Actor**: Represents a user or an external entity interacting with the system.



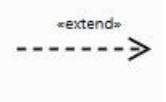
- **Use Case**: Represented by an oval. It describes a function performed by the system.



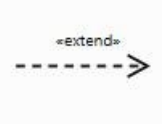
- **System Boundary**: Represented by a rectangle enclosing the use cases.



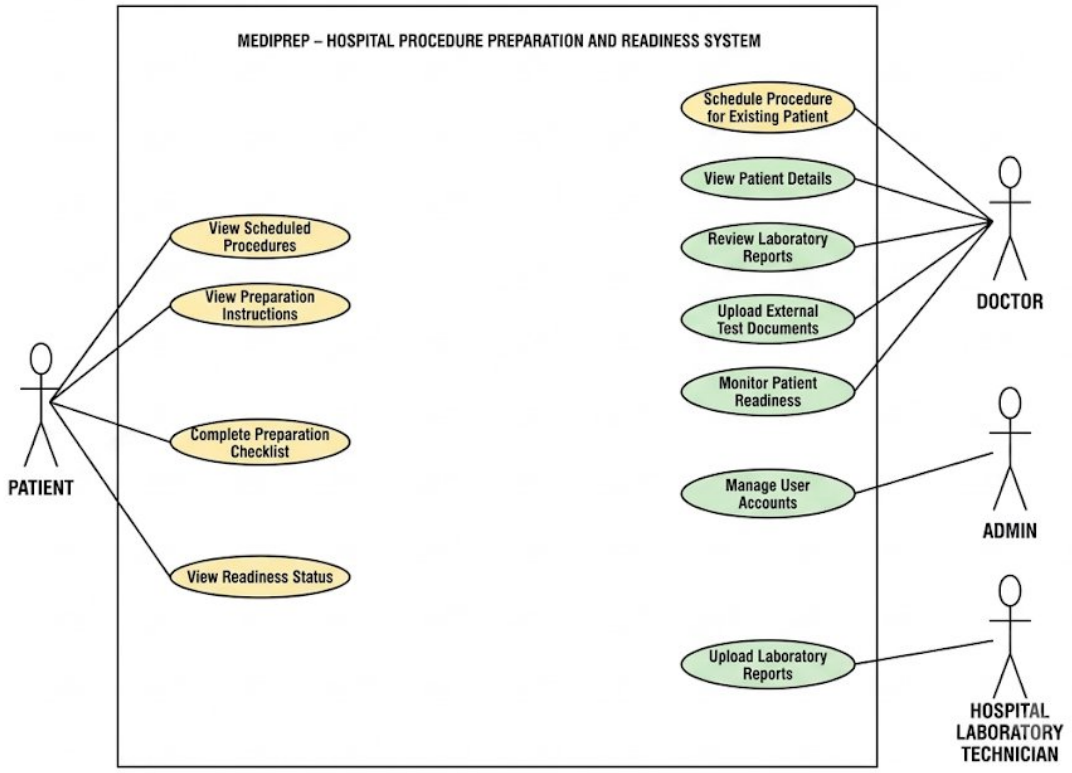
- **Include Relationship**: Represented by a dashed arrow labelled «include». It indicates that one use case always includes the behaviour of another use case.



- **Extend Relationship**: Represented by a dashed arrow labelled «extend». It indicates optional or conditional behaviour added to another use case.



Use Case Diagram



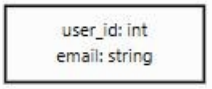
5.2 Class Diagram

A UML class diagram shows the structure of a system by displaying its classes, their attributes, methods, and the relationships between them. It helps the team understand how the system is organized and how components interact.

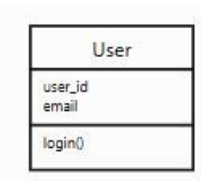
- Represents classes, attributes, methods, and relationships within a system.
- Helps developers understand, communicate, and document the software structure more effectively.

Class Diagram Symbols

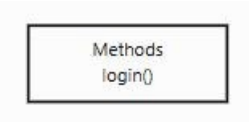
- **Attribute**: Represents the data or properties of a class.



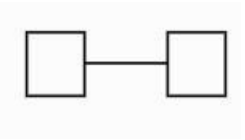
- **Class**:Represented by a rectangle divided into three compartments: class name, attributes, and methods.



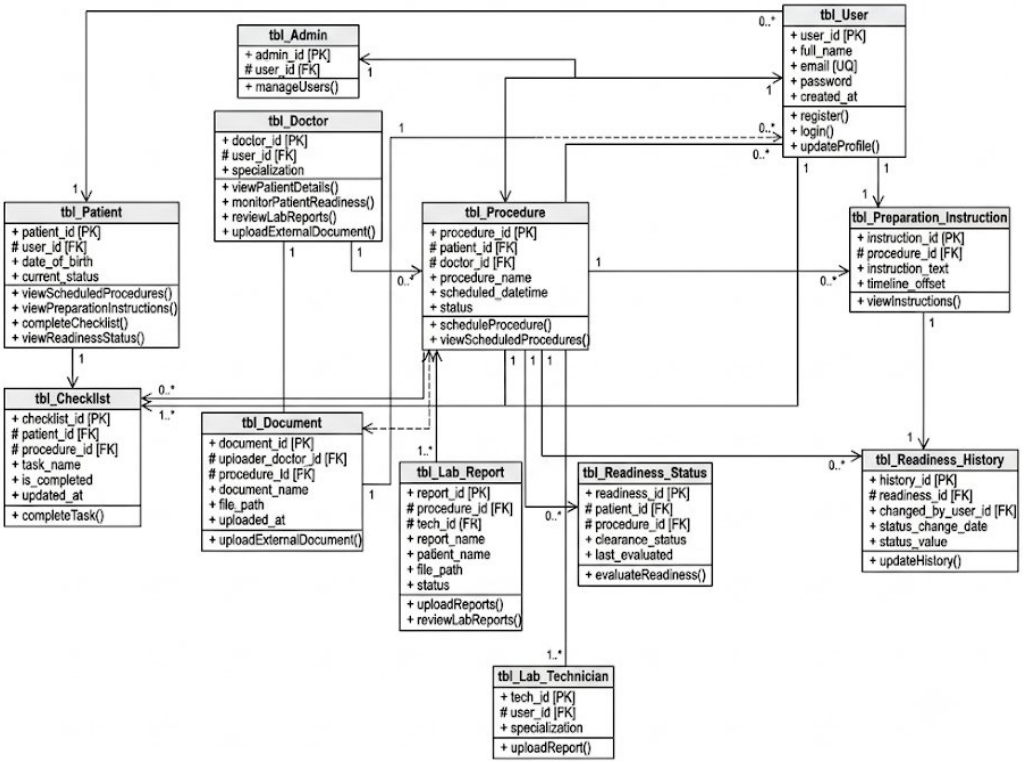
- **Method / Operation**: Represents the behaviour or functions performed by a class.



- **Association**: Represented by a solid line connecting two classes to indicate the relationship between them.



Class Diagram



SYSTEM DESIGN

6.1 SYSTEM DESIGN

System design is the process of defining the architecture, modules, interfaces, and database structure of a system to satisfy the specified requirements. It explains how the proposed system is developed and how its different components work together as a single application. The design phase translates the requirements identified during system analysis into a practical solution.In MediPrep, the system design focuses on developing a web-based application for managing hospital procedure scheduling, patient preparation checklists, laboratory reports, and readiness monitoring. The design includes input screens, output screens, and database tables required for the proper functioning of the system. Each component is designed to provide a user-friendly interface, organized data management, and efficient access to information. The following sections describe the input design, output design, and database design of the system.

6.2 Input Design

Input design is the process of creating user-friendly forms through which data is entered into the system. It plays an important role in ensuring that the information provided by users is accurate, complete, and suitable for processing. Proper input design reduces data-entry errors and makes the application easier to use.In MediPrep, input forms are designed for different user roles, including Admin, Doctor, Patient, and Hospital Laboratory Technician. These forms allow authorized users to enter and manage the information required for hospital procedure preparation.The system includes login forms, patient account details, procedure scheduling forms, preparation checklist inputs, and laboratory report upload forms. Doctors can select existing patients and enter procedure details, while authorized laboratory technicians can upload official reports. Patients can access and track their preparation checklist requirements.Input fields are organized clearly, and suitable validation checks are used to reduce incorrect or incomplete entries. Role-based access ensures that users can perform only the functions permitted for their roles. The input design aims to provide simple navigation, consistent form layouts, and clear messages to help users interact with the system effectively.

6.3 Output Design

Output design refers to the presentation of processed information to users in a clear and meaningful format. The output should be accurate, relevant, and easy to understand so that users can access the information required for their tasks.

In MediPrep, the output screens display user-specific information based on the user's role. The Admin can view user records, while doctors can view scheduled procedures, laboratory reports, checklist completion status, and readiness information. Patients can view their scheduled procedures, preparation instructions, and checklist status. Hospital Laboratory Technicians can access the functions available to them for laboratory report management.

The system displays incomplete preparation requirements and predefined rule-based alerts to help doctors identify cases that require attention. The output design uses organized dashboards, clear labels, and status indicators to make important information easy to understand.

6.4 Table Design

The efficiency of a database application depends on the design of its tables, fields, and relationships. A table stores data in rows and columns, with each row representing a record and each column representing a particular attribute. Primary keys uniquely identify records, while foreign keys establish relationships between tables.

The main objective of database design is to maintain data accuracy, reduce duplication, ensure data integrity, and retrieve information efficiently. MediPrep uses a MySQL database to store user details, patient information, scheduled procedures, preparation checklists, laboratory reports, and readiness alerts.

The main tables used in the project are:

1. tbl_users
2. tbl_patient_profiles
3. tbl_doctor_profiles
4. tbl_scheduled_procedures
5. tbl_checklist_items
6. tbl_uploaded_reports
7. tbl_procedure_readiness
8. tbl_risk_alerts

1. tbl_users

Description: This table stores the details of users registered in the system.

Field Name	Data Type	Description
user_id	INT (PK)	Unique user ID
name	VARCHAR(100)	User name
email	VARCHAR(100)	User email
password	VARCHAR(255)	Hashed password
role	VARCHAR(30)	Admin, Doctor, Patient, or Laboratory Technician
phone	VARCHAR(20)	Contact number
status	VARCHAR(20)	Account status

2. tbl_patient_profiles

Description: This table stores additional information about patients.

Field Name	Data Type	Description
patient_id	INT (PK)	Unique patient ID
user_id	INT (FK)	Linked user ID
date_of_birth	DATE	Patient's date of birth
gender	VARCHAR(20)	Patient's gender
address	TEXT	Patient's address

3. tbl_doctor_profiles

Description: This table stores additional details of doctors.

Field Name	Data Type	Description
doctor_id	INT (PK)	Unique doctor ID
user_id	INT (FK)	Linked user ID
specialization	VARCHAR(100)	Medical specialization
qualification	VARCHAR(100)	Doctor's qualification

4.tbl_scheduled_procedures

Description: This table stores details of procedures scheduled for patients.

Field Name	Data Type	Description
scheduled_procedure_id	INT (PK)	Unique scheduled procedure ID
patient_id	INT (FK)	Linked patient ID
doctor_id	INT (FK)	Linked doctor ID
procedure_name	VARCHAR(150)	Name of the procedure
scheduled_date	DATE	Procedure date
scheduled_time	TIME	Procedure time
status	VARCHAR(30)	Procedure status

5. tbl_checklist_items

Description: This table stores preparation checklist requirements for scheduled procedures.

Field Name	Data Type	Description
checklist_id	INT (PK)	Unique checklist ID
scheduled_procedure_id	INT (FK)	Linked scheduled procedure
item_name	VARCHAR(150)	Preparation requirement
status	VARCHAR(30)	Completion status
remarks	TEXT	Additional remarks

6. tbl_uploaded_reports

Description: This table stores laboratory report details uploaded by authorized users.

Field Name	Data Type	Description
report_id	INT (PK)	Unique report ID
patient_id	INT (FK)	Linked patient ID
scheduled_procedure_id	INT (FK)	Linked procedure ID
uploaded_by	INT (FK)	User who uploaded the report
report_name	VARCHAR(150)	Name of the report
file_path	VARCHAR(255)	Stored report file path
uploaded_at	DATETIME	Upload date and time

7. tbl_procedure_readiness

Description: This table stores the readiness status of patients for scheduled procedures.

Field Name	Data Type	Description
readiness_id	INT (PK)	Unique readiness ID
scheduled_procedure_id	INT (FK)	Linked scheduled procedure
readiness_status	VARCHAR(30)	Current readiness status
completion_percentage	DECIMAL(5,2)	Checklist completion percentage
updated_at	DATETIME	Last update date and time

8. tbl_risk_alerts

Description: This table stores alerts generated when critical preparation requirements are incomplete.

Field Name	Data Type	Description
alert_id	INT (PK)	Unique alert ID
scheduled_procedure_id	INT (FK)	Linked scheduled procedure
alert_message	TEXT	Description of the alert
severity	VARCHAR(20)	Alert severity
status	VARCHAR(30)	Alert status
created_at	DATETIME	Alert creation date and time

TOOLS AND PLATFORM

7.1 Front End: HTML, CSS, JavaScript, Bootstrap

The front end of MediPrep is developed using HTML, CSS, JavaScript, and Bootstrap. HTML provides the structure of web pages, while CSS is used to improve their appearance and layout. JavaScript adds interactivity to the application, and Bootstrap helps create responsive layouts and user-friendly components. Together, these technologies provide a simple interface for administrators, doctors, patients, and laboratory technicians.

7.2 Back End: Python

Python is used to develop the backend of MediPrep. It handles application logic, processes user requests, manages procedure scheduling, and communicates with the database. Python's simple syntax and extensive libraries make it suitable for developing web applications.

7.3 Framework: Flask

Flask is a lightweight Python web framework used to develop MediPrep. It handles URL routing, request processing, and communication between the user interface and the database. Its flexibility supports the development of a modular application with role-based access and patient readiness monitoring.

7.4 Platform: Visual Studio Code

Visual Studio Code (VS Code) is used as the primary code editor for developing and managing the MediPrep project. It supports Python, HTML, CSS, and JavaScript and provides useful features such as syntax highlighting, extensions, and debugging tools. It helps organize project files and simplify application development.

7.5 Database: MySQL

MySQL is used to store and manage the data required by MediPrep. It maintains user accounts, patient profiles, scheduled procedures, preparation checklists, laboratory reports, and readiness information. Primary and foreign keys help maintain relationships between tables and support accurate data retrieval.

TESTING

8.1 Unit Testing

Unit testing is the process of testing individual modules of an application separately to verify that each module functions correctly. It helps identify errors at an early stage of development and ensures that each component produces the expected results. In this testing, individual functions, input fields, and database operations are checked independently before integrating the modules into the complete application.

In MediPrep, unit testing is performed on the user management, procedure scheduling, preparation checklist, laboratory report management, and readiness monitoring modules. The login module is tested using valid and invalid credentials to verify authentication and input validation. The procedure scheduling module is checked to ensure that procedures are assigned to the correct existing patient and that the required details are stored properly. The preparation checklist module is tested to verify that completed and incomplete requirements are recorded correctly.

The laboratory report management module is tested to ensure that authorized users can upload reports and that the stored report information is available for review. The readiness monitoring module is checked to verify whether incomplete critical requirements are identified according to the predefined rules. By testing each module separately, errors can be detected and corrected before the modules are integrated into the complete MediPrep application.

8.2 Integration Testing

Integration testing is the process of testing the interaction between different modules after they are combined into a single system. It ensures that data is transferred correctly between the modules and that their functions work together without errors.

In MediPrep, integration testing verifies the connection between user accounts, patient profiles, and scheduled procedures. It also checks whether the preparation checklist is correctly associated with the scheduled procedure and whether checklist completion is reflected in the patient's readiness status. Laboratory report uploads and their availability for doctor review are also tested. This testing helps identify errors in communication between modules and ensures the smooth functioning of the application.

8.3 System Testing

System testing is carried out to evaluate the complete application after integrating all its modules. The main purpose is to verify whether the developed system satisfies the specified requirements and performs its functions correctly.In MediPrep, system testing covers user login, role-based access, patient management,procedure scheduling, preparation checklist

tracking,laboratory report management, and readiness monitoring. The application is tested using different inputs to check whether it produces the expected outputs. It also verifies that users can access only the functions permitted for their roles and that scheduled procedures and related preparation details are displayed correctly. System testing helps ensure the overall reliability and proper functioning of MediPrep.

8.1.1 Acceptance Testing

The system was tested by a small client community to see if the program met the requirements defined the analysis stage. It was fond to be satisfactory. In this phase, the system is fully tested by the client community against the requirements defined in analysis and design stages, corrections are made as required, and the production system is built. User acceptance of the system is key factor for success of the system.

8.1.2 Validation Testing

Data validation is the process of testing the accuracy of data. A set of rules we can apply to a control to specify the type and range of data that can enter. It can be used to display error alert when users enter incorrect values in to a form. Now performing validation testing in system Centralized Social Welfare by undergoing validation for each tool and the validation succeeded when the software function in a manner that can be reasonably accepted, by the user.

8.1.3 Black box Testing

Testing Knowing the specified function that a product has been designed to perform, test can be conducted that demonstrates each function that is fully operational, at the same time searching for errors in each function. Black Box testing focuses on functional requirement of the software.

8.1.4 White box Testing

Testing Knowing the internal working of a product test can be conducted to ensure that “all gears mesh” that is internal operation performs according to specification and all internal components have been adequately exercised.

8.4 Sample Test Cases

TC No.	Test Steps	Expected Result	Actual Result	Status
1	Run the application and navigate to the login screen.	The login screen should display fields for email address and password, along with a login button.	The login screen is displayed with email and password fields and a login button.	Pass
2	Enter an invalid email address and password and click the login button.	An error message should indicate that the login credentials are invalid.	An error message is displayed for invalid login credentials.	Pass
3	Enter valid login credentials and click the login button.	The user should successfully log in and navigate to the appropriate dashboard.	The user logs in successfully and the appropriate dashboard is displayed.	Pass
4	Enter a valid email address and leave the password field empty. Click the login button.	A validation message should request the password.	A validation message is displayed requesting the password.	Pass
5	Leave both email and password fields empty and click the login button.	A validation message should request the required login details.	A validation message is displayed requesting the required login details.	Pass
6	Leave the email field empty, enter a valid password, and click the login button.	A validation message should request the email address.	A validation message is displayed requesting the email address.	Pass
7	Schedule a procedure for an existing patient through the doctor dashboard.	The procedure should be saved and linked to the correct patient.	The scheduled procedure is saved and linked to the selected patient.	Pass
8	Open the patient dashboard after a procedure has been scheduled.	The scheduled procedure and its preparation checklist should be displayed.	The scheduled procedure and preparation checklist are displayed.	Pass
9	Leave a critical preparation checklist item incomplete.	The system should identify the incomplete requirement and display an appropriate alert.	The incomplete requirement and corresponding alert are displayed.	Pass
10	Upload a laboratory report using an authorized laboratory technician account.	The report should be stored and made available for authorized review.	The report is stored and available for authorized review.	Pass

SYSTEM IMPLEMENTATION

9. SYSTEM IMPLEMENTATION

9.1 Implementation

Implementation is the process of converting the system design into a working application. It includes developing the required modules, connecting them with the database, testing their functionality, and preparing the system for use. Proper implementation is essential to ensure that the application works effectively and meets the requirements of its users.The implementation phase of MediPrep involves developing the user management, procedure scheduling, preparation checklist, laboratory report management, and readiness monitoring modules. Python Flask is used for backend processing, MySQL is used for database management, and HTML, CSS, and JavaScript are used to develop the user interface.During implementation, the different modules are connected to ensure the proper flow of information between them. Role-based access is provided for administrators, doctors, patients, and hospital laboratory technicians according to their responsibilities. Patient details, scheduled procedures, checklist information, and laboratory reports are maintained in the database. The application is tested to identify errors and verify that the implemented functions operate according to the specified requirements. Successful implementation helps provide an organized and reliable system for managing hospital procedure preparation.

9.2 Problem Statement

Proper preparation before hospital procedures is essential to ensure that patients complete the required tests, documentation, and medical instructions. However, managing procedure schedules, preparation checklists, and laboratory reports through manual records or separate systems can make it difficult to monitor patient readiness.

Doctors and hospital staff may need to check multiple records to identify missing reports and incomplete preparation requirements. Patients may also face difficulties in accessing procedure details and remembering the instructions they must follow before their scheduled procedures. These issues can increase manual work, create communication difficulties, and cause delays in preparation.

Therefore, there is a need for an organized system like MediPrep that brings procedure scheduling, preparation checklist tracking, laboratory report management, and readiness monitoring together on a single platform. This helps doctors identify incomplete requirements and supports better coordination among hospital personnel and patients.

9.3 Problem Definition

The main problem addressed by MediPrep is the difficulty of maintaining accurate and organized information about patient preparation before scheduled hospital procedures. Manual tracking of procedure details, laboratory reports, and preparation requirements can lead to incomplete records and difficulties in identifying patients who have not completed the necessary preparations.

Doctors need a convenient way to schedule procedures for existing patients, review laboratory reports, and monitor checklist completion. Patients also need easy access to their scheduled procedures and preparation instructions. Without a centralized platform, maintaining and retrieving this information can require additional time and effort.

MediPrep aims to address these problems by providing a web-based system for procedure scheduling, preparation tracking, and laboratory report management. Its predefined rule-based alerts help identify incomplete critical requirements that need attention. The system supports organized record-keeping, improves coordination among authorized users, and helps doctors monitor patient readiness before scheduled procedures.

CONCLUSION

10 . CONCLUSION

The MediPrep – Hospital Procedure Preparation and Readiness System is developed to simplify the management of patient preparation before hospital procedures. The application integrates procedure scheduling, preparation checklist tracking, laboratory report management, and readiness monitoring into a single web-based platform.The system provides separate functions for administrators, doctors, patients, and hospital laboratory technicians according to their assigned roles. Doctors can schedule procedures for existing patients, review laboratory reports, and monitor preparation progress. Patients can view their scheduled procedures and preparation instructions, while authorized laboratory technicians can upload official laboratory reports.The project aims to reduce dependence on manual record-keeping and make patient preparation information easier to manage. The use of predefined rule-based alerts helps identify incomplete preparation requirements and cases that need further attention. Thus, MediPrep provides an organized approach to hospital procedure preparation and supports better coordination among hospital personnel.

FUTURE ENHANCEMENTS

MediPrep has been designed so that additional features can be introduced in the future according to hospital requirements. Although the current system supports procedure scheduling, preparation tracking, laboratory report management, and readiness monitoring, further improvements can enhance its usability and flexibility.
Future enhancements may include the following features:

- **Reminder Notifications:** Notifications can be introduced to remind patients about upcoming procedures and pending preparation requirements.
- **Mobile Application:** The system can be extended into a mobile application to provide easier access through smartphones.
- **Enhanced Reports:** Additional reports can be generated to summarize scheduled procedures, checklist completion, and patient readiness status.
- **Improved Checklist Management:** More procedure-specific checklist templates can be added to support different types of surgeries and diagnostic tests.
- **Extended Monitoring:** Additional features can be introduced to help doctors review previous procedure records and monitor preparation progress more effectively.

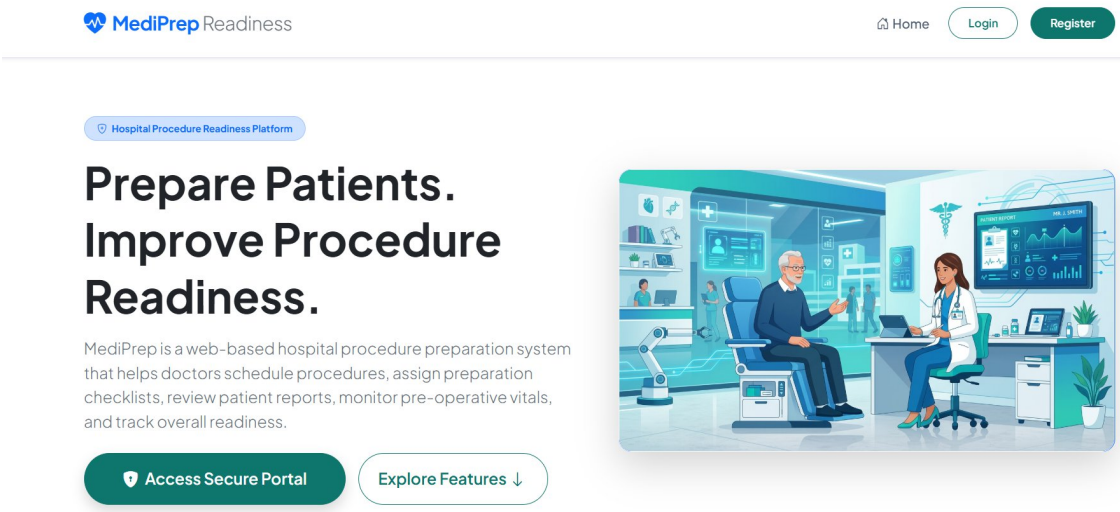
These enhancements can improve the functionality of MediPrep and make it more adaptable to changing hospital requirements. The system provides a foundation for further development in hospital procedure preparation and readiness management.

APPENDICES

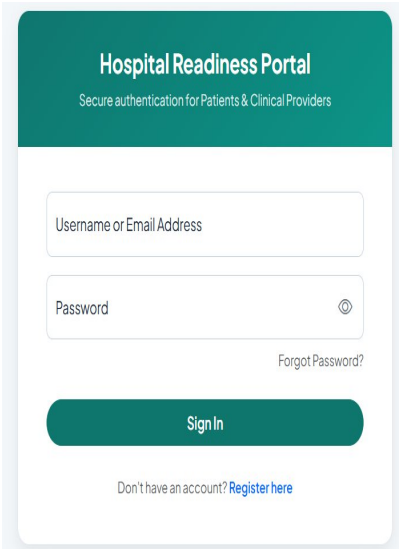
11. APPENDICES

APPENDICES A:Sample Screens

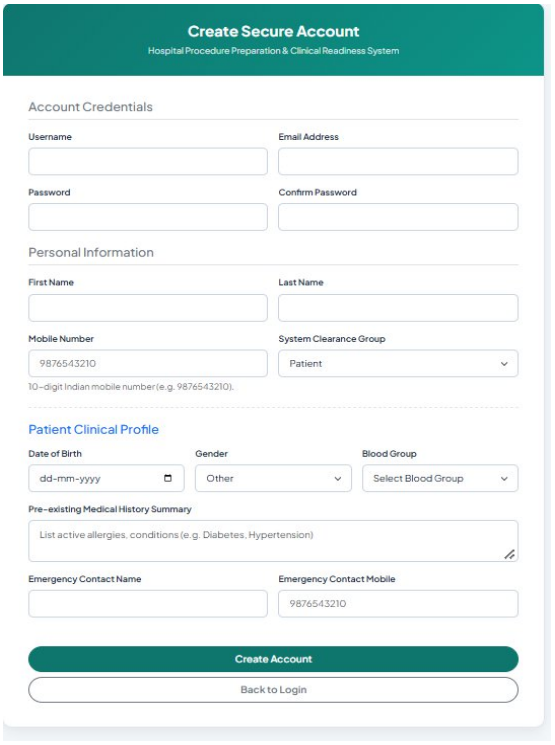
Home Page



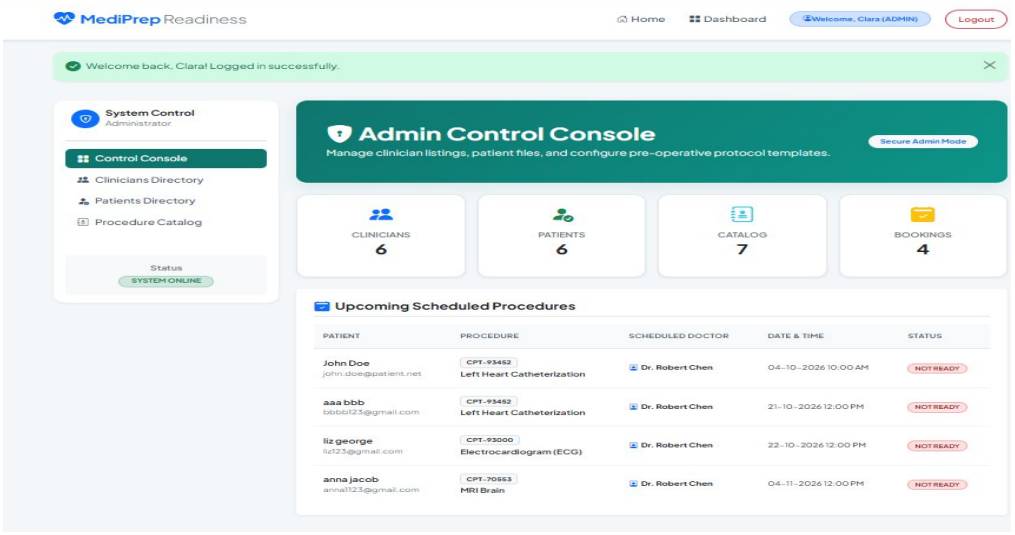
Login Page



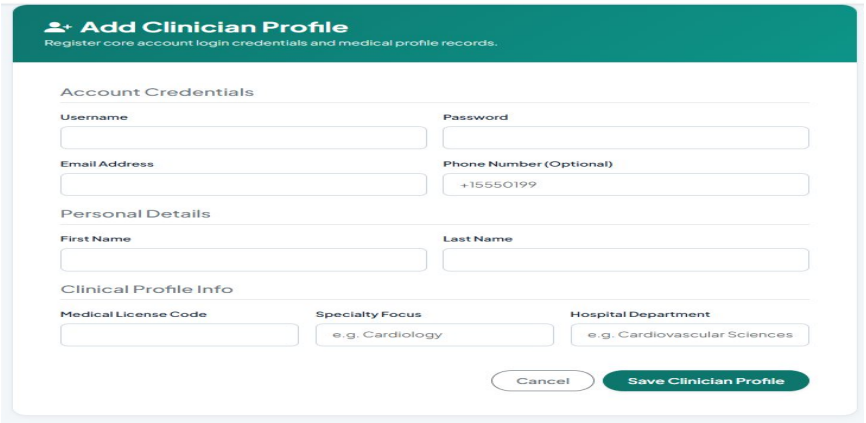
Registration Page



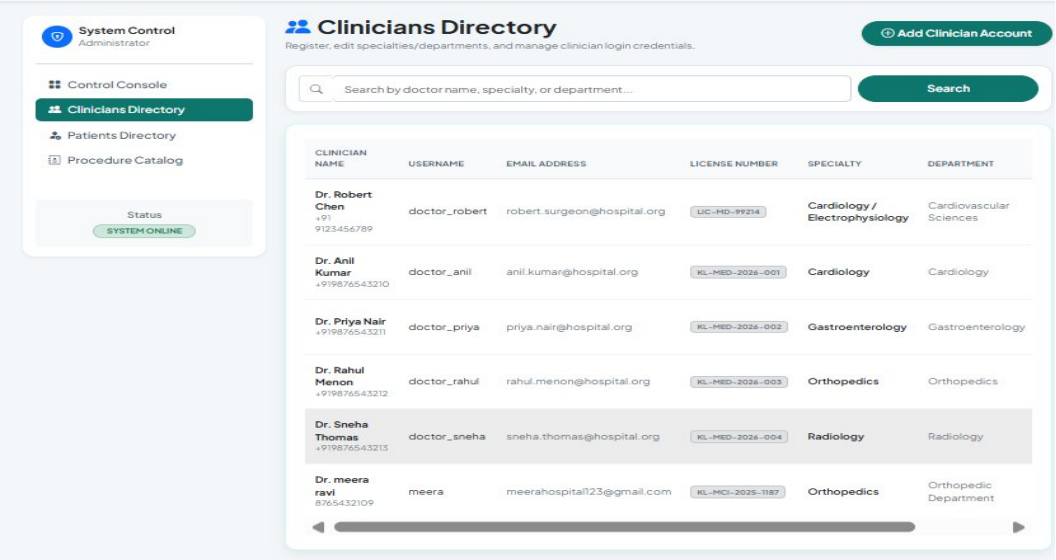
Admin Dashboard



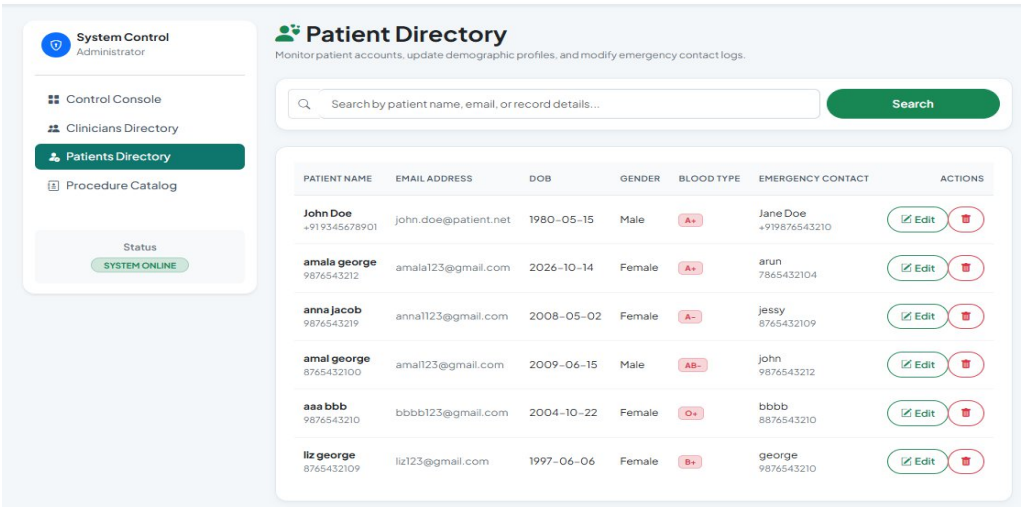
Add Clinician Profile



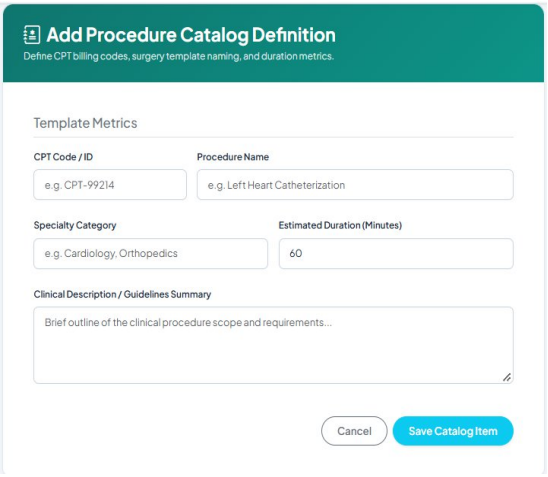
View Clinician Profile



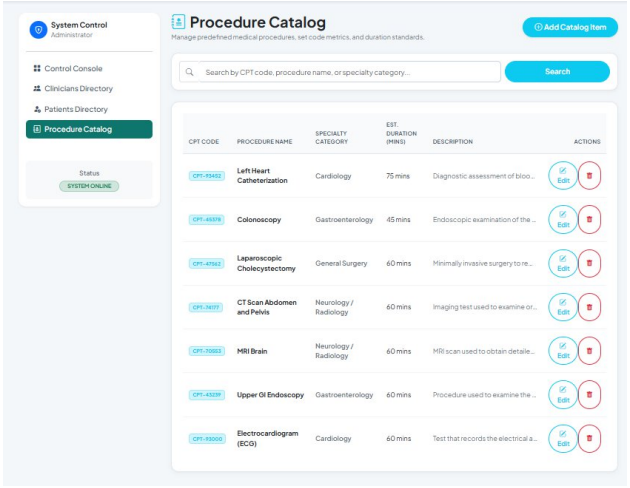
Patient Directory



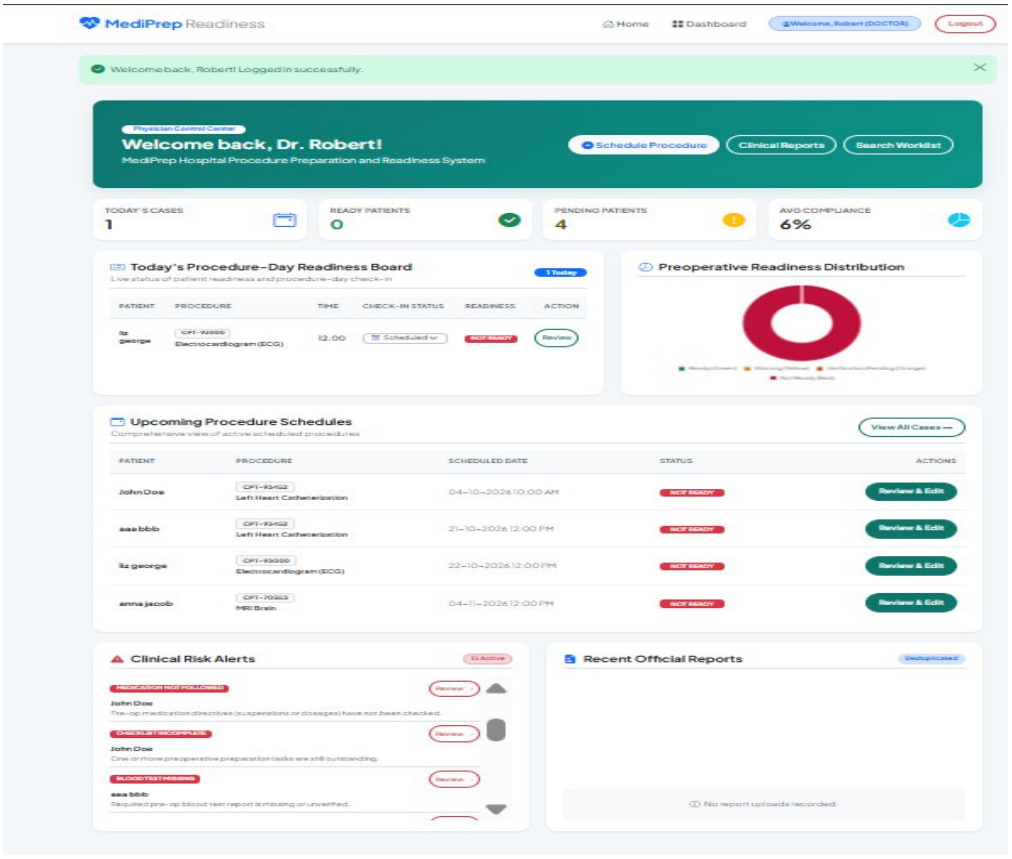
Add Procedure Catalog



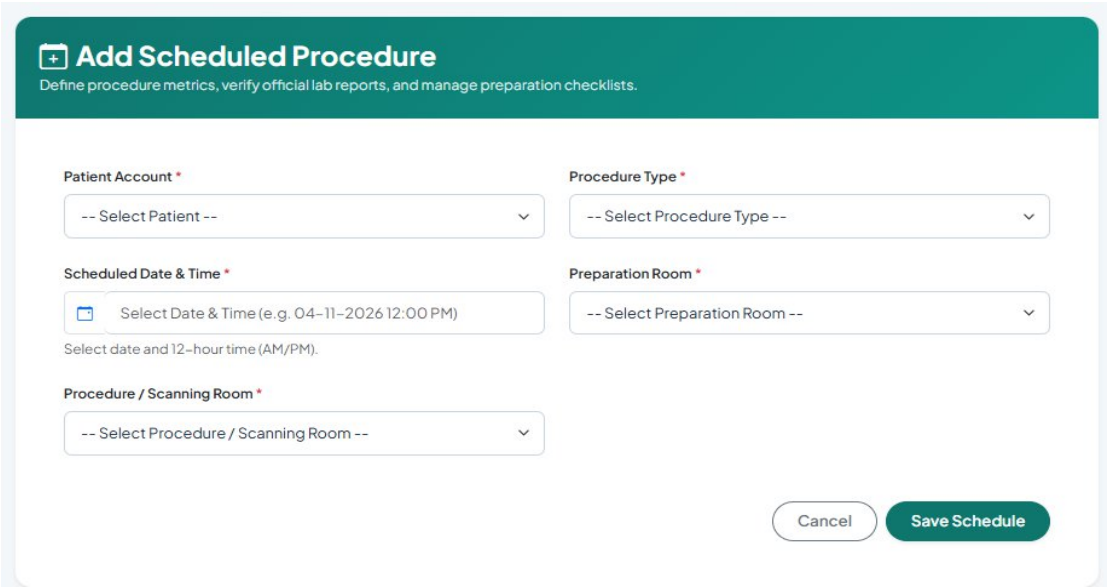
View Procedure Catalog



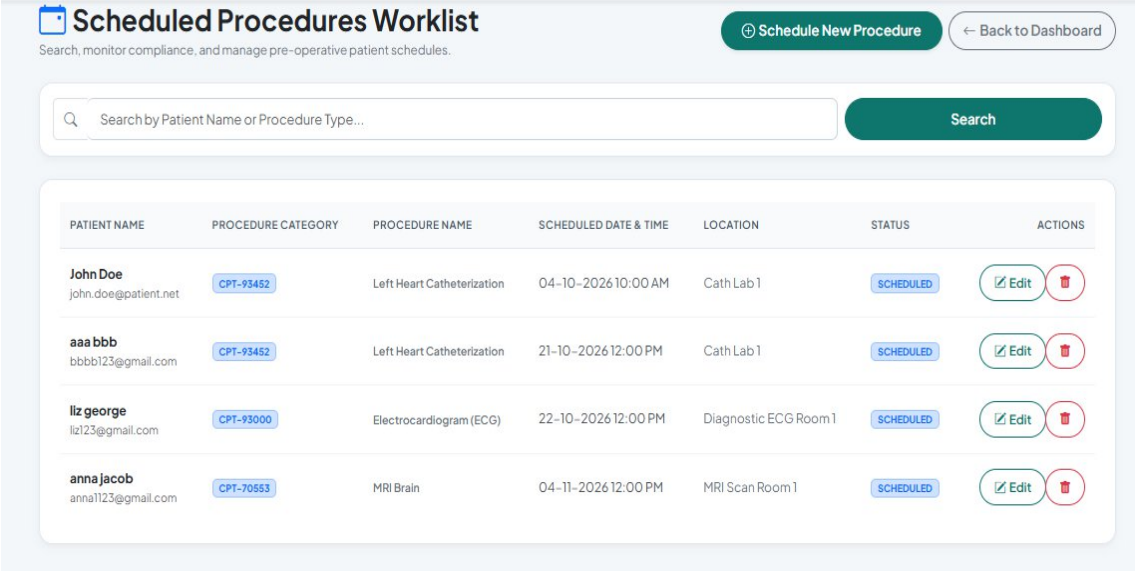
Doctor Dashboard



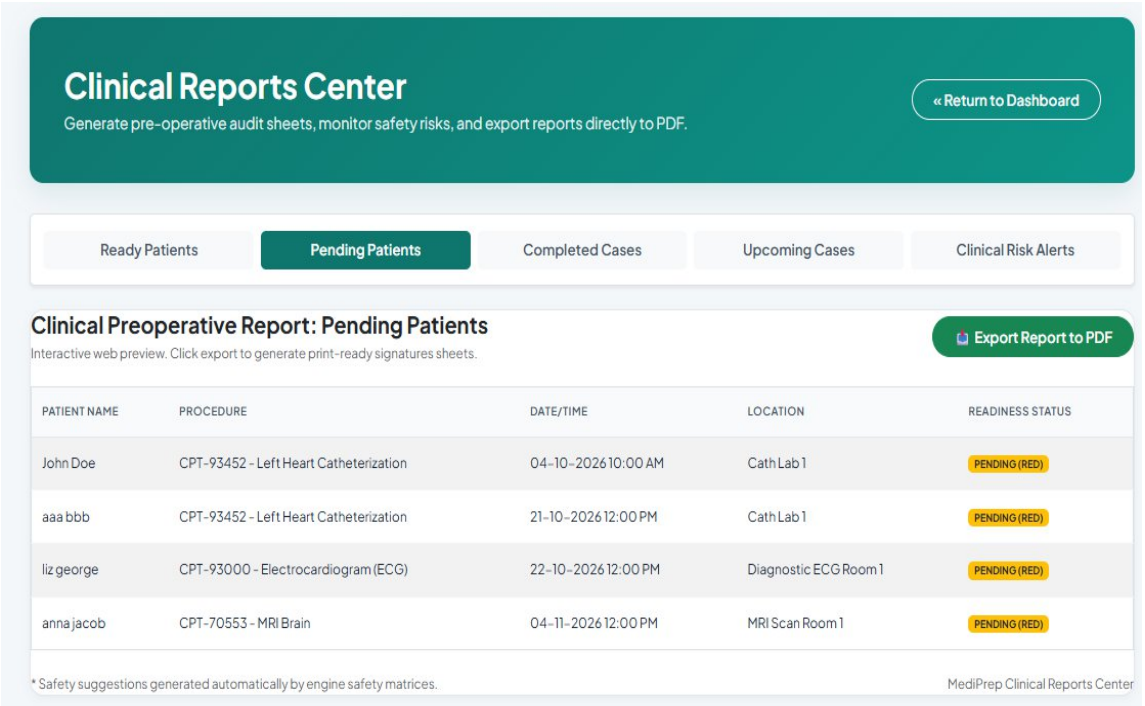
Add Scheduled Procedure



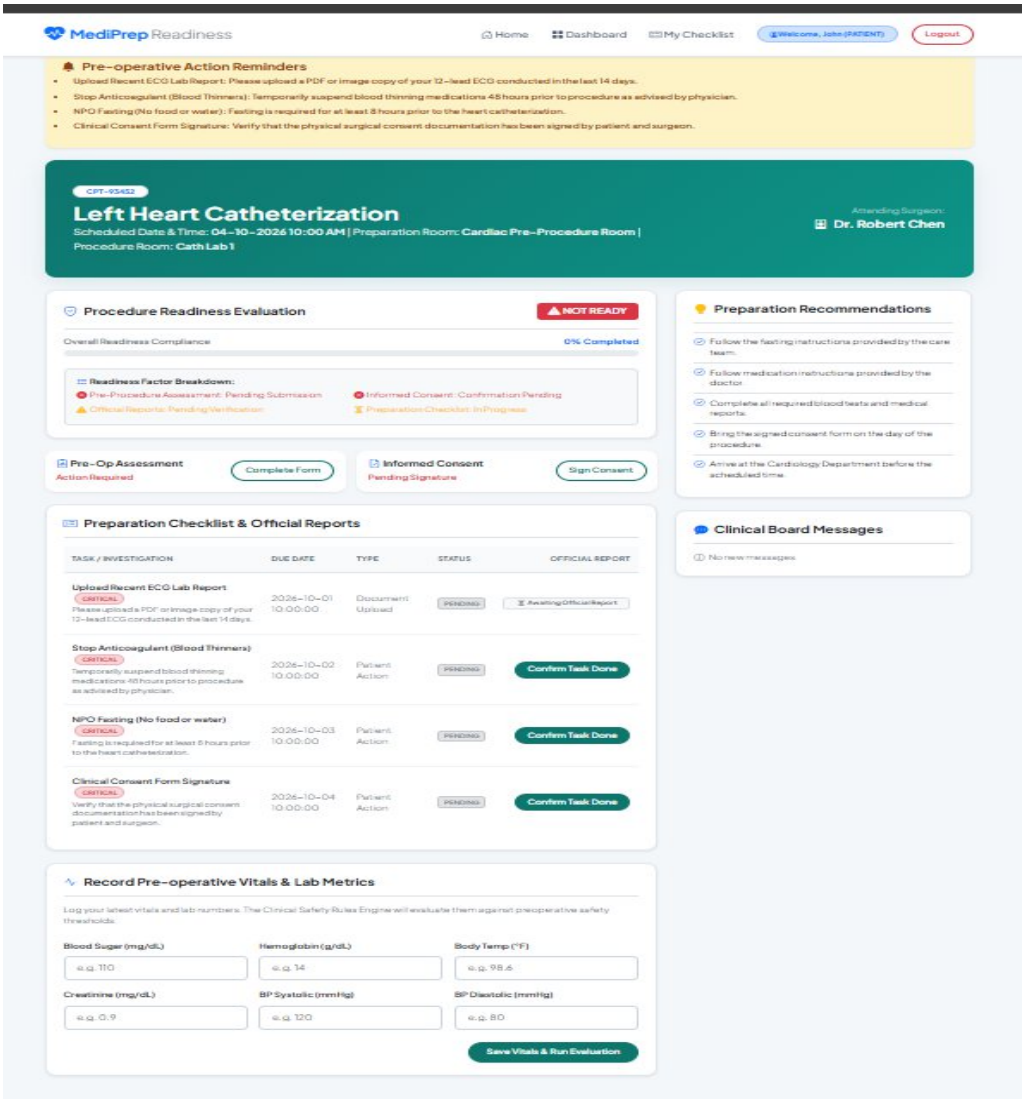
View Scheduled Procedures



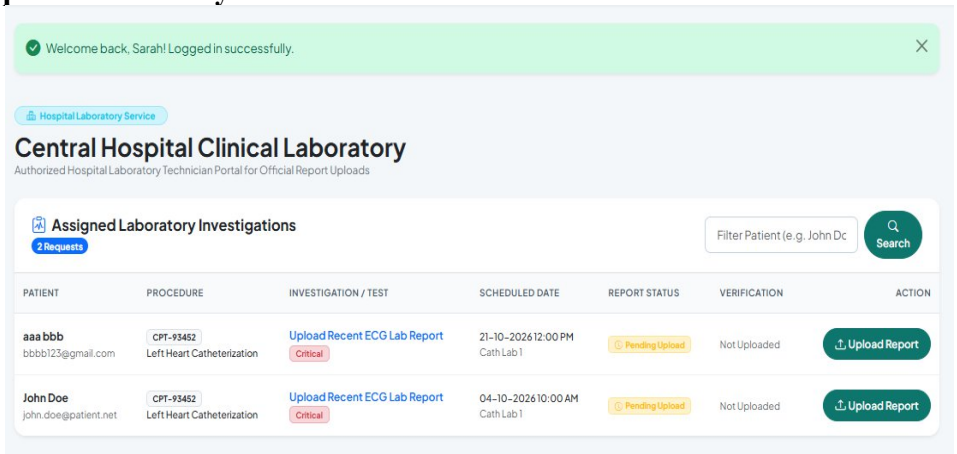
Clinical Reports



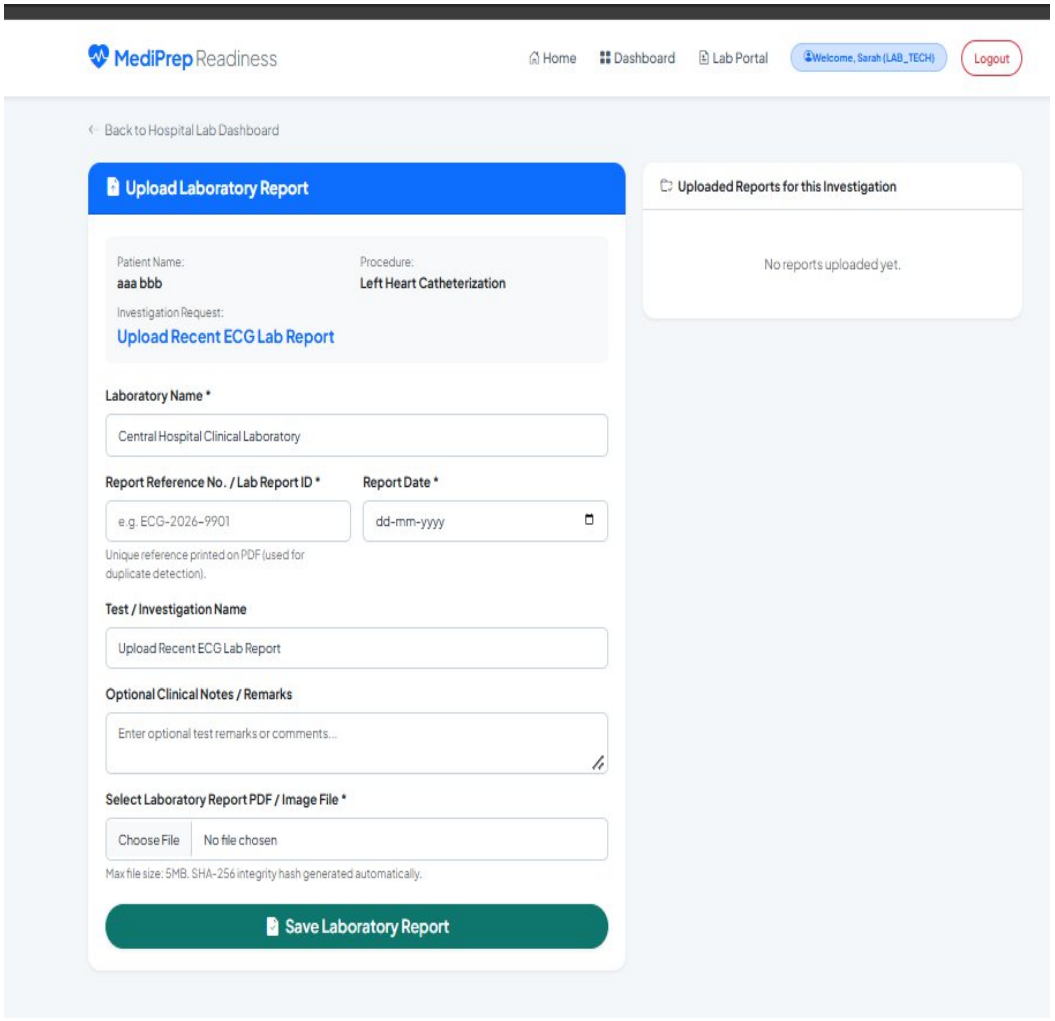
Patient Dashboard



Hospital Laboratory Technician Dashboard



Upload Report



APPENDICES B:Sample Code

```
import os
import re
import json
import hashlib
from datetime import datetime, date, timedelta
from functools import wraps
from werkzeug.utils import secure_filename
from flask import (
    Flask, render_template, request, redirect,
    url_for, session, flash, abort, current_app, jsonify
)
import mysql.connector
from db import get_db_connection
#
=============================================================================
===
# ROUTE PROTECTION DECORATORS
#
=============================================================================
===
def login_required(f):
    @wraps(f)
    def decorated_function(*args, **kwargs):
        if 'user_id' not in session:
            flash("Please log in to access this page.", "warning")
            return redirect(url_for('login'))
        return f(*args, **kwargs)
    return decorated_function
def role_required(*roles):
    def decorator(f):
        @wraps(f)
        def decorated_function(*args, **kwargs):
            if 'user_id' not in session:
                flash("Please log in to access this page.", "warning")
                return redirect(url_for('login'))
            user_role = session.get('role')
            if user_role not in roles:
                abort(403)
```

```
  return f(*args, **kwargs)
        return decorated_function
    return decorator
#
=============================================================================
===
# HOME PAGE
#
=============================================================================
===
@app.route('/')
def home():
    return render_template('home.html')
#
=============================================================================
===
# USER LOGIN
#
=============================================================================
===
@app.route('/login', methods=['GET', 'POST'])
def login():
    if 'user_id' in session:
        return redirect(url_for('dashboard'))
    if request.method == 'POST':
        email = request.form.get('email', '').strip()
        password = request.form.get('password', '')
        # EMAIL REQUIRED
        if not email:
            flash("Email address is required.", "danger")
            return render_template('login.html')
        # PASSWORD REQUIRED
  if not password:
            flash("Password is required.", "danger")
            return render_template('login.html')
        # VERIFY CREDENTIALS
        conn = get_db_connection()
        cursor = conn.cursor(dictionary=True)
        cursor.execute("SELECT * FROM tbl_User WHERE email = %s", (email,))
        user = cursor.fetchone()
        if not user or user['password'] != password:
            cursor.close()
```

```
            conn.close()

 flash("Invalid email or password.", "danger")
            return render_template('login.html')
        # CREATE SESSION
        session['user_id'] = user['user_id']
        session['full_name'] = user['full_name']
        session['email'] = user['email']
        session['role'] = user['role']
        cursor.close()
        conn.close()
        # ROLE-BASED REDIRECT
        if user['role'] == 'admin':
            return redirect(url_for('admin_dashboard'))
        elif user['role'] == 'doctor':
            return redirect(url_for('doctor_dashboard'))
        elif user['role'] == 'patient':
            return redirect(url_for('patient_dashboard'))
        elif user['role'] == 'lab_tech':
            return redirect(url_for('lab_dashboard'))
        else:
            session.clear()
            flash("Invalid user role. Contact administrator.", "danger")
            return render_template('login.html')
    return render_template('login.html')
#
=============================================================================
===
# LOGOUT
#
=============================================================================
===
@app.route('/logout')
def logout():
    session.clear()
    flash("You have been logged out.", "info")
    return redirect(url_for('login'))
#
=============================================================================
===
# DOCTOR: SCHEDULE PROCEDURE
#
=============================================================================
```

```
@app.route('/doctor/schedule/add', methods=['GET', 'POST'])
@login_required
@role_required('doctor')
def schedule_procedure():
    doctor_id = session['user_id']
    conn = get_db_connection()
    cursor = conn.cursor(dictionary=True)
    if request.method == 'POST':
        patient_id = request.form.get('patient_id')
        procedure_name = request.form.get('procedure_name', '').strip()
        scheduled_datetime = request.form.get('scheduled_datetime')
        # ADVANCE NOTICE VALIDATION (24 HOURS)
        sched_time = datetime.strptime(scheduled_datetime, '%Y-%m-%dT%H:%M')
        if sched_time < datetime.now() + timedelta(hours=24):
            flash("Procedure must be scheduled at least 24 hours in advance.", "danger")
            return redirect(url_for('schedule_procedure'))
        # INSERT SCHEDULE RECORD
    insert_query = """
            INSERT INTO tbl_Procedure (patient_id, doctor_id, procedure_name,
scheduled_datetime, status)
            VALUES (%s, %s, %s, %s, 'scheduled')
        """
        cursor.execute(insert_query, (patient_id, doctor_id, procedure_name,
scheduled_datetime))
        procedure_id = cursor.lastrowid
        # INITIALIZE DEFAULT PREPARATION CHECKLIST
        default_tasks = [
            ("Fasting 8 hours prior to procedure", "patient_action"),
            ("Blood Routine & Sugar Profile", "lab_investigation"),
            ("Signed Informed Consent", "patient_action")
        ]
        for task_name, task_type in default_tasks:
            cursor.execute("""
                INSERT INTO tbl_Checklist (patient_id, procedure_id, task_name, is_completed)
                VALUES (%s, %s, %s, FALSE)
            """, (patient_id, procedure_id, task_name))
        # INITIALIZE READINESS STATUS
        cursor.execute("""
            INSERT INTO tbl_Readiness_Status (patient_id, procedure_id, clearance_status,
last_evaluated)
            VALUES (%s, %s, 'red', NOW())
```

```
    """, (patient_id, procedure_id))
        conn.commit()
        cursor.close()
        conn.close()
        flash("Procedure scheduled and checklist initialized successfully.", "success")
        return redirect(url_for('doctor_dashboard'))
    cursor.execute("SELECT user_id, full_name FROM tbl_User WHERE role = 'patient'")
    patients = cursor.fetchall()
    cursor.close()
    conn.close()
    return render_template('doctor/schedule_form.html', patients=patients)

#
==========================================================================
===
# PATIENT: DASHBOARD & CHECKLIST
#
==========================================================================
===
@app.route('/patient/dashboard')
@login_required
@role_required('patient')
def patient_dashboard():
    patient_id = session['user_id']
    conn = get_db_connection()
    cursor = conn.cursor(dictionary=True)
    cursor.execute("""
        SELECT p.*, r.clearance_status
        FROM tbl_Procedure p
        LEFT JOIN tbl_Readiness_Status r ON p.procedure_id = r.procedure_id
        WHERE p.patient_id = %s ORDER BY p.scheduled_datetime DESC
    """, (patient_id,))
    procedures = cursor.fetchall()
    checklist = []
    if procedures:
        active_proc = procedures[0]
        cursor.execute("""
            SELECT * FROM tbl_Checklist
            WHERE procedure_id = %s
        """, (active_proc['procedure_id'],))
        checklist = cursor.fetchall()
```

```
    cursor.close()
    conn.close()
    return render_template('patient/dashboard.html', procedures=procedures,
checklist=checklist)
@app.route('/patient/checklist/complete/<int:checklist_id>', methods=['POST'])
@login_required
@role_required('patient')
def complete_checklist_item(checklist_id):
    conn = get_db_connection()
    cursor = conn.cursor()
    cursor.execute("""
        UPDATE tbl_Checklist
        SET is_completed = TRUE, updated_at = NOW()
        WHERE checklist_id = %s
    """, (checklist_id,))
    conn.commit()
    cursor.close()
    conn.close()
    flash("Checklist item updated.", "success")
    return redirect(url_for('patient_dashboard'))
#
==========================================================================
===
# LAB TECHNICIAN: REPORT UPLOAD
#
==========================================================================
===
@app.route('/lab/upload', methods=['GET', 'POST'])
@login_required
@role_required('lab_tech')
def upload_lab_report():
    if request.method == 'POST':
        patient_id = request.form.get('patient_id')
        procedure_id = request.form.get('procedure_id')
        report_name = request.form.get('report_name', '').strip()
        file = request.files.get('report_file')
        if not file or file.filename == '':
            flash("Please choose a valid report file.", "danger")
            return redirect(url_for('upload_lab_report'))
        filename = secure_filename(file.filename)
        file_bytes = file.read()
```

```
 file_hash = hashlib.sha256(file_bytes).hexdigest()[:12]
        file.seek(0)
  unique_filename = f"report_{procedure_id}_{file_hash}_{filename}"
        upload_path = os.path.join(current_app.config['UPLOAD_FOLDER'], unique_filename)
        file.save(upload_path)
        conn = get_db_connection()
        cursor = conn.cursor()
        cursor.execute("""
            INSERT INTO tbl_Laboratory_Report
            (patient_id, tech_id, report_name, upload_date, file_path, status)
            VALUES (%s, %s, %s, NOW(), %s, 'submitted')
        """, (patient_id, session['user_id'], report_name, unique_filename))
        conn.commit()
        cursor.close()
        conn.close()
        flash("Laboratory report uploaded successfully.", "success")
        return redirect(url_for('lab_dashboard'))
    return render_template('lab/upload.html')
#
==========================================================================
===
# DOCTOR: EXTERNAL DOCUMENT UPLOAD
#
==========================================================================
===
@app.route('/doctor/upload_document', methods=['POST'])
@login_required
@role_required('doctor')
def upload_external_document():
    patient_id = request.form.get('patient_id')
    procedure_id = request.form.get('procedure_id')
    doc_type = request.form.get('doc_type', 'External Lab Report')
    file = request.files.get('doc_file')
    if not file or file.filename == '':
        flash("File is required for external report upload.", "danger")
        return redirect(url_for('doctor_dashboard'))
    filename = secure_filename(file.filename)
    unique_name = f"ext_{procedure_id}_{filename}"
    file.save(os.path.join(current_app.config['UPLOAD_FOLDER'], unique_name))
    conn = get_db_connection()
    cursor = conn.cursor()
```

```
 cursor.execute("""
        INSERT INTO tbl_Document (patient_id, doc_type, file_path, upload_timestamp)
        VALUES (%s, %s, %s, NOW())
    """, (patient_id, doc_type, unique_name))
    conn.commit()
    cursor.close()
    conn.close()
    flash("External report uploaded and attached by doctor.", "success")
    return redirect(url_for('doctor_dashboard'))
#
==========================================================================
===
# READINESS EVALUATION & SAFETY RULES ENGINE
#
==========================================================================
===
def evaluate_patient_readiness(procedure_id):
    conn = get_db_connection()
    cursor = conn.cursor(dictionary=True)
    # 1. CHECK PENDING TASKS
    cursor.execute("""
        SELECT COUNT(*) as pending_count
        FROM tbl_Checklist
        WHERE procedure_id = %s AND is_completed = FALSE
    """, (procedure_id,))
    pending_tasks = cursor.fetchone()['pending_count']
    # 2. CHECK VERIFIED LAB REPORTS
    cursor.execute("""
        SELECT COUNT(*) as unverified_count
FROM tbl_Laboratory_Report
        WHERE patient_id = (SELECT patient_id FROM tbl_Procedure WHERE procedure_id
= %s)
          AND status != 'verified'
    """, (procedure_id,))
    unverified_reports = cursor.fetchone()['unverified_count']
    # 3. RULE-BASED STATUS DETERMINATION
    if pending_tasks == 0 and unverified_reports == 0:
        new_status = 'green'
    elif pending_tasks > 0 and unverified_reports == 0:
        new_status = 'yellow'
    else:
```

```
new_status = 'red'
  cursor.execute("""
     UPDATE tbl_Readiness_Status
     SET clearance_status = %s, last_evaluated = NOW()
     WHERE procedure_id = %s
  """, (new_status, procedure_id))
  conn.commit()
  cursor.close()
  conn.close()
  return new_status
```

APPENDICES C: Hardware and Software Requirements

- Hardware Requirements

Processor	:	Intel Core i5 or equivalent
Random Access Memory	:	4GBor above
Hard Disk/SSD	:	240GB or above
Monitor	:	Color Monitor
Keyboard	:	Standard
Video	:	800 X 600 or higher

- Software Requirements

Operating System	:	Windows 11 or higher
Front End	:	HTML,CSS,Javascript
Back End	:	Python
Framework	:	Flask
Database	:	MySQL
Operating System	:	Windows 11
Development Environment	:	Visual Studio Code
Documentation Tool	:	MS Word
Browser	:	Google Chrome or any modern web browser

APPENDICES D: Bibliography

Book References:

- Jose, J. *Taming Python by Programming*. Khanna Publishing House. Retrieved from Google Books.

Publications:

- Pallets Projects. *Flask Documentation*. Available at: www.flask.palletsprojects.com.
- Python Software Foundation. *Python Documentation*. Available at: www.python.org/doc.
- Oracle. *MySQL Documentation*. Available at: www.dev.mysql.com/doc.

Website References:

- Retrieved from www.stackoverflow.com
- Retrieved from www.w3schools.com
- Retrieved from www.github.com
- Retrieved from www.python.org
- Retrieved from www.tutorialspoint.com
- Retrieved from www.javatpoint.com
- Retrieved from flask.palletsprojects.com
- Retrieved from dev.mysql.com